

Operational Models para Verificação de Brokers MQTT com ESBMC

Apresentação inicial

Gustavo Lima Willian Jean Paulo Gomes

Maio de 2026

Queremos usar **verificação formal** para encontrar vulnerabilidades em programas C usados em IoT.

O caso escolhido é o **Eclipse Mosquitto**, um broker MQTT escrito em C.

Tese do trabalho

Investigar a extensão do ESBMC com *operational models* para APIs de rede, permitindo verificar módulos de software MQTT real.

Por que Mosquitto e MQTT?

- MQTT é um protocolo leve muito usado em Internet das Coisas.
- O broker é o ponto central: recebe, roteia e entrega mensagens.
- O Mosquitto é open source, amplamente usado e escrito em C.
- Bugs no broker podem afetar toda a rede de dispositivos.

Esse contexto junta segurança, C, IoT e verificação formal em um problema real.

O ESBMC já verifica propriedades importantes em C:

- buffer overflow;
- integer overflow;
- null pointer dereference;
- use-after-free;
- memory leak.

Mas software de rede usa muitas APIs externas: `socket`, `accept`, `recv`, `send`, `select`, `poll`.

Sem modelos adequados dessas APIs, a verificação de programas reais fica limitada.

O que são *operational models*?

Operational model é uma implementação abstrata de uma função externa.

Em vez de verificar a implementação real do sistema operacional, o ESBMC usa um modelo menor e controlado.

Exemplo

Um modelo de `recv()` pode dizer: “o buffer recebe até `len` bytes não determinísticos”.

Isso preserva o comportamento relevante para segurança sem explodir o espaço de estados.

Pergunta

Como estender o ESBMC com *operational models* para APIs de rede, de forma a detectar vulnerabilidades em módulos de brokers MQTT escritos em C?

Objetivo geral

Investigar e avaliar modelos operacionais que permitam a análise formal de módulos do Mosquitto com ESBMC.

Resultados esperados:

- Catálogo das APIs que bloqueiam a verificação direta do Mosquitto.
- Conjunto de *operational models* para APIs de rede (`socket`, `recv`, `send`, `select`).
- Avaliação comparativa: ESBMC sem modelos vs. ESBMC com modelos.
- Estudo de caso sobre a viabilidade de reproduzir CVEs históricos do Mosquitto.

Abordagem:

Análise do código-fonte → Seleção de APIs → Implementação incremental → Validação comparativa → Estudo de caso

Exemplo: parser do *Remaining Length*

O campo *Remaining Length* do MQTT usa codificação variável. Um parser incorreto pode sofrer overflow aritmético.

```
1 uint32_t read_remaining_length(const uint8_t *buf, size_t buflen)
2 {
3     uint32_t multiplier = 1;
4     uint32_t value = 0;
5     uint8_t byte;
6     size_t pos = 0;
7
8     do {
9         if (pos >= buflen) return 0;
10        byte = buf[pos++];
11        value += (byte & 127) * multiplier;
12        multiplier *= 128;
13    } while ((byte & 128) != 0);
```


Execução real do ESBMC

Comando usado:

```
esbmc slides\code\mqtt_bof_example.c --unwind 8 --no-unwinding-assertions --unsigned-overflow-check
```

[Counterexample]

```
State 19 file slides\code\mqtt_bof_example.c line 16 column 9 function read_remaining_length thread 0
-----
```

Violated property:

```
file slides\code\mqtt_bof_example.c line 16 column 9 function read_remaining_length
arithmetic overflow on mul
!overflow(" ", (unsigned int)((signed int)byte & 127), multiplier)
```

VERIFICATION FAILED

ESBMC version 8.2.0 64-bit x86_64 windows

- Mapear quais APIs externas impedem a verificação direta do Mosquitto.
- Propor modelos operacionais para um subconjunto de APIs de rede.
- Comparar ESBMC original versus ESBMC com os novos modelos.
- Investigar a viabilidade de reproduzir CVEs históricos como estudo de caso.

A contribuição não é criar outro analisador, mas tornar o ESBMC mais aplicável a software de rede real.

Obrigado.